

Fiche de révision — Systèmes temps réel (S8)

Méthodes clés pour traiter n'importe quel exercice de temps réel

1. Notions de base

- **Système temps réel** : la correction dépend non seulement du résultat mais aussi du *moment* où il est produit.
- **Temps réel dur (hard)** : dépasser une échéance (deadline) est une faute grave/catastrophique (ex. commande de vol).
- **Temps réel mou (soft)** : un dépassement dégrade la qualité de service sans être catastrophique (ex. streaming vidéo).
- **Tâche périodique** : caractérisée par (C_i, T_i, D_i) = temps d'exécution pire cas (WCET), période, échéance (souvent $D_i = T_i$ si non précisé).
- **WCET** (Worst-Case Execution Time) : durée d'exécution maximale garantie d'une tâche, utilisée pour tous les calculs d'ordonnabilité.

2. Ordonnancement — les deux politiques de référence

Algorithme	Principe	Priorités
Rate Monotonic (RM)	priorité fixe, inversement proportionnelle à la période : plus la période est courte, plus la priorité est haute	fixes, assignées une fois pour toutes
Earliest Deadline First (EDF)	à chaque instant, on exécute la tâche dont l'échéance absolue est la plus proche	dynamiques, recalculées en continu

Méthode d'exercice « dérouler un ordonnancement » : construire une frise temporelle. À chaque instant (ou à chaque arrivée/fin de tâche), déterminer la tâche prête de plus haute priorité (RM : priorité fixe donnée ; EDF : comparer les échéances absolues courantes = date d'arrivée + D_i) et l'exécuter jusqu'à préemption ou fin.

3. Tests d'ordonnabilité

Test suffisant de Liu & Layland (RM)

$U = \sum (C_i / T_i) \leq n \cdot (2^{1/n} - 1)$, où n = nombre de tâches. Si l'inégalité est vraie, le système est ordonnable avec RM (condition suffisante mais pas nécessaire : si elle est fausse, on ne peut rien conclure directement, il faut un test exact).

Test exact / condition nécessaire et suffisante (EDF)

Avec EDF, la condition nécessaire ET suffisante (pour $D_i = T_i$) est simplement : $U = \sum (C_i / T_i) \leq 1$. C'est le résultat le plus important à retenir sur EDF : il est optimal parmi les algorithmes à priorités dynamiques.

Méthode d'exercice « vérifier l'ordonnabilité »

- 1) Calculer le taux d'utilisation $U = \sum (C_i / T_i)$ pour toutes les tâches.
- 2) Pour EDF : comparer U à 1 directement (test exact).
- 3) Pour RM : comparer U à la borne de Liu & Layland ; si le test échoue, ne pas conclure « non ordonnable » sans faire un test exact (ex. analyse par temps de réponse).

Analyse par temps de réponse (test exact pour RM)

Temps de réponse R_i d'une tâche $i = C_i + \sum$ (sur les tâches j de priorité plus haute) $\text{ceil}(R_j / T_j) \cdot C_j$, résolu par itération à point fixe en partant de $R_i = C_i$. La tâche est ordonnable si $R_i \leq D_i$.

4. Inversion de priorité et solutions

- **Inversion de priorité** : une tâche de haute priorité est bloquée en attendant une ressource partagée détenue par une tâche de basse priorité, elle-même préemptée par une tâche de priorité intermédiaire → la tâche prioritaire attend indirectement une tâche moins prioritaire qu'elle.
- **Protocole d'héritage de priorité** : la tâche qui détient la ressource hérite temporairement de la priorité de la tâche bloquée la plus prioritaire, le temps de libérer la ressource.
- **Protocole à priorité plafond (priority ceiling)** : chaque ressource a une priorité plafond = priorité max des tâches pouvant l'utiliser ; une tâche ne peut entrer en section critique que si sa priorité est supérieure à tous les plafonds actuellement verrouillés → borne le blocage à une seule section critique de plus basse priorité.

5. Tâches apériodiques et sporadiques

- **Apériodique** : arrivée non prévisible, pas de contrainte stricte d'échéance en général.
- **Sporadique** : arrivée non prévisible mais avec une période minimale garantie entre deux arrivées consécutives (permet de borner la charge pire cas comme une tâche périodique).
- **Serveur de tâches apériodiques** (ex. serveur à scrutation/polling, serveur différé, serveur sporadique) : réserve une capacité de temps CPU dédiée pour traiter rapidement les tâches apériodiques sans compromettre l'ordonnabilité des tâches périodiques.

6. Conception d'un système temps réel (RTOS)

- **Ordonnanceur préemptif à priorités** : la tâche la plus prioritaire prête s'exécute toujours en premier, en interrompant une tâche moins prioritaire en cours.
- **Sémaphores/mutex** pour la synchronisation et l'exclusion mutuelle sur les ressources partagées — attention aux interblocages (deadlocks) et à l'inversion de priorité (cf. section 4).
- **Interruptions** : traitées en dehors de l'ordonnancement normal des tâches, avec leur propre priorité matérielle ; une routine d'interruption doit rester très courte (déléguer le traitement long à une tâche).
- **Latence** à distinguer : latence d'interruption (détection → début de traitement) et latence d'ordonnancement (tâche prête → tâche éeue).

7. Méthode générale pour un exercice de synthèse

- 1) Identifier pour chaque tâche : C_i , T_i , D_i (et vérifier si $D_i = T_i$ ou différent).
- 2) Choisir/identifier l'algorithme d'ordonnancement demandé (RM ou EDF).
- 3) Calculer U et appliquer le test adapté (borne de Liu & Layland pour RM, $U \leq 1$ pour EDF, ou temps de réponse si test exact demandé).
- 4) Si demandé, dérouler la frise d'exécution sur au moins un hyperpériode (PPCM des périodes) pour visualiser/valider.
- 5) Si des ressources partagées interviennent, vérifier l'absence d'inversion de priorité non bornée (protocole d'héritage / priorité plafond).

8. Pièges classiques en examen

- Utiliser le test de Liu & Layland comme s'il était nécessaire ET suffisant : il n'est que suffisant pour RM.
- Oublier que EDF n'a un test aussi simple ($U \leq 1$) que lorsque $D_i = T_i$; sinon il faut une analyse plus fine (ex. demande cumulée).
- Confondre priorité fixe (RM) et priorité dynamique (EDF) lors du déroulé d'une frise temporelle.
- Oublier de recalculer l'hyperpériode (PPCM des T_i) pour être sûr d'avoir simulé un cycle complet du système.

- Ne pas borner le blocage dû à l'inversion de priorité alors que l'énoncé mentionne une ressource partagée — c'est souvent la question piège.